

Regla de la cadena y descenso por gradiente en redes neuronales

La regla de la cadena es una herramienta fundamental en el entrenamiento de redes neuronales profundas. Permite calcular cómo afecta cada peso o sesgo al valor final de la función de pérdida. Como una red neuronal está formada por varias capas encadenadas, la salida depende de una secuencia de operaciones: combinaciones lineales, funciones de activación y, por último, la evaluación del error. La regla de la cadena descompone ese proceso en derivadas locales más simples, lo que hace posible calcular gradientes y ajustar los parámetros del modelo de forma eficiente.

1. Conceptos fundamentales

1. Funciones compuestas o anidadas. En una red neuronal, la salida de una capa se convierte en la entrada de la siguiente. Por eso, el resultado final puede entenderse como una composición de funciones.
2. Derivadas parciales y gradiente. La derivada parcial indica cómo cambia una función cuando se modifica una sola variable, manteniendo las demás constantes. El conjunto de todas esas derivadas forma el gradiente.
3. Retropropagación. Es el algoritmo que aplica la regla de la cadena para calcular el gradiente del error con respecto a cada parámetro de la red, capa por capa, desde la salida hasta la entrada.

2. Fórmula general de la regla de la cadena

Si una variable depende de otra a través de varias funciones intermedias, la derivada total puede obtenerse multiplicando las derivadas de cada etapa. Si $y = f(g(h(x)))$, entonces:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}$$

3. Aplicación de la regla de la cadena en redes neuronales

En una neurona de la capa l , primero se calcula la combinación lineal de las entradas y después se aplica una función de activación. De forma general:

$$z_j^{(l)} = \sum_i w_{ij}^{(l)} a_i^{(l-1)} + b_j^{(l)}, \quad a_j^{(l)} = \sigma(z_j^{(l)})$$

Para conocer cómo cambia la pérdida L cuando se modifica un peso $w_{ij}^{(l)}$, se aplica la regla de la cadena a lo largo de las operaciones que conectan ese peso con la salida final. Una forma clara de expresarlo es la siguiente:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \frac{\partial L}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}}$$

Donde:

- L es la función de pérdida.
- $w_{ij}^{(l)}$ es el peso que conecta la neurona i de la capa $(l-1)$ con la neurona j de la capa l .
- $z_j^{(l)}$ es la entrada lineal a la función de activación de la neurona j .
- $a_j^{(l)}$ es la salida de la neurona j después de aplicar la activación.

Como la derivada de $z_j^{(l)}$ respecto a $w_{ij}^{(l)}$ es precisamente $a_i^{(l-1)}$, el gradiente puede escribirse de forma compacta así:

$$\frac{\partial L}{\partial w_{ij}^{(l)}} = \delta_j^{(l)} a_i^{(l-1)}$$

Esta expresión resume una idea clave: el gradiente de un peso depende del error local de la neurona y de la activación que recibe desde la capa anterior.

4. Retropropagación: cómo se calculan los gradientes

La retropropagación utiliza la regla de la cadena para propagar el error desde la capa de salida hacia las capas anteriores. El procedimiento básico es el siguiente:

1. Se realiza la propagación hacia delante para obtener la salida de la red.
2. Se calcula la función de pérdida comparando la salida predicha con el valor real.
3. Se propaga el error hacia atrás y se calcula el gradiente de cada peso y cada sesgo.
4. Se actualizan los parámetros para reducir la pérdida.

En una neurona sigmoide sencilla con pérdida logística, el ciclo de entrenamiento puede representarse de la siguiente manera:

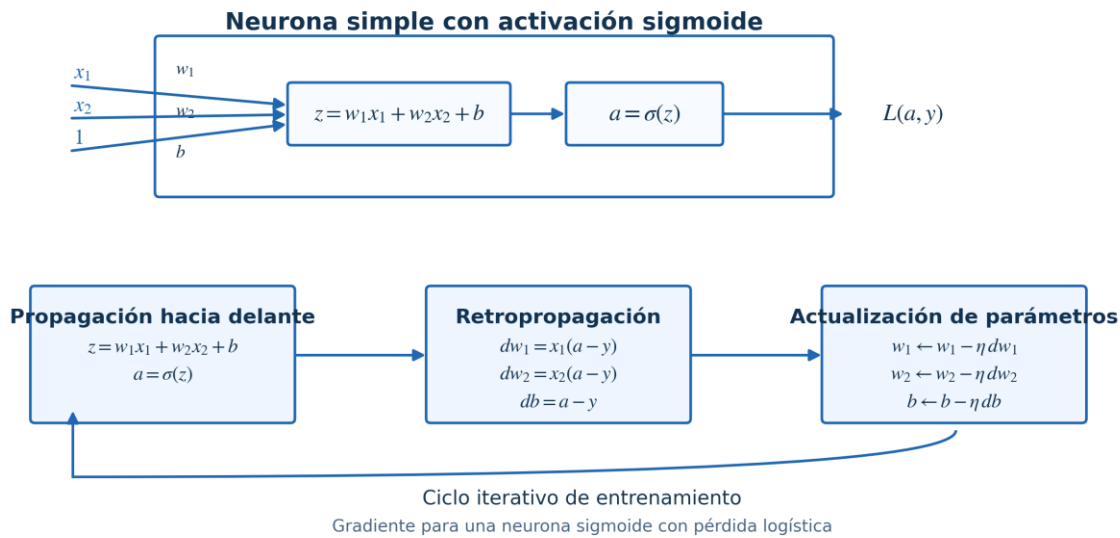


Figura 1. Ciclo básico de entrenamiento de una neurona simple con activación sigmoide.

Nota: las expresiones $dw_1 = x_1(a - y)$, $dw_2 = x_2(a - y)$ y $db = a - y$ corresponden al caso de una neurona sigmoide con pérdida logística o entropía cruzada binaria.

5. Descenso por gradiente (Gradient Descent)

El descenso por gradiente es un algoritmo de optimización cuyo objetivo es encontrar el mínimo de la función de pérdida. Para ello, utiliza el gradiente y actualiza los parámetros en la dirección opuesta a dicho gradiente.

La actualización general de los pesos se expresa como:

$$W \leftarrow W - \eta \nabla_W L$$

1. Se calcula el gradiente de la función de pérdida con respecto a los parámetros del modelo.
2. Se actualizan los parámetros en la dirección opuesta al gradiente para reducir el error.
3. El proceso se repite hasta que la pérdida deja de disminuir de forma significativa o se alcanza un criterio de parada.

La tasa de aprendizaje η controla el tamaño del paso en cada actualización. Elegirla bien es fundamental para que el entrenamiento sea estable y eficiente.

- Si η es demasiado grande, el algoritmo puede sobrepasar el mínimo o incluso divergir.
- Si η es demasiado pequeña, el aprendizaje será muy lento.

Una analogía útil es imaginar que se desciende por una colina en busca del punto más bajo. El gradiente indica la dirección de mayor subida; por tanto, moverse en la dirección contraria permite bajar.

6. Relación entre la función de coste y el descenso por gradiente

La función de coste mide qué tan bien o qué tan mal está funcionando el modelo en un momento dado. El descenso por gradiente utiliza la información proporcionada por la derivada de esa función para ajustar los parámetros. A medida que el entrenamiento avanza, la función de coste debería disminuir hasta acercarse a un mínimo.

7. Ejemplo numérico sencillo

Supongamos que queremos minimizar la siguiente función:

$$L(W) = (W - 3)^2 \quad \Rightarrow \quad \frac{dL}{dW} = 2(W - 3)$$

Si elegimos como valor inicial $W = 0$ y una tasa de aprendizaje $\eta = 0.1$, las primeras iteraciones quedan así:

- Iteración 1: $W_1 = 0 - 0.1 \cdot 2(0 - 3) = 0.6$
- Iteración 2: $W_2 = 0.6 - 0.1 \cdot 2(0.6 - 3) = 1.08$
- Iteración 3: $W_3 = 1.08 - 0.1 \cdot 2(1.08 - 3) = 1.464$

Si se continúa iterando, el valor de W se aproxima progresivamente a 3, que es el mínimo de la función.

8. Conclusión

En redes neuronales, la regla de la cadena permite calcular de forma ordenada cómo influye cada parámetro en el error final. La retropropagación aplica esa idea para obtener los gradientes capa por capa, y el descenso por gradiente utiliza esos gradientes para actualizar pesos y sesgos. En conjunto, estos dos mecanismos constituyen la base matemática del aprendizaje en redes neuronales.